

COURSE CODE/TITLE:	CPE 212 Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	08/29/2026	DATE SUBMITTED:	08/30/2026
NAME:	Interno, Jerald San M.	INSTRUCTOR:	Engr. Jimlord

ACTIVITY NO. 5.1 Implement Ansible roles in playbooks

1. PROCEDURE OUTPUT

Task 1: Use when command for different distributions

1. In the local machine, make sure you are in the local repository directory (CPE232_yourname). Issue the command `git pull`. When prompted, enter the correct passphrase or password. Describe what happened when you issue this command.

```
jerald@workstation:~/ansible-lab$ git pull
Already up to date.
```

Did something happen? Why?

- The terminal output reads Already up to date. Since the local branch matches the latest commit state on the remote GitHub repository, so no new files or changes were downloaded.

2. Edit the inventory file and add the IP address of the Centos VM. Issue the command we used to execute the playbook (the one we used in the last activity): `ansible-playbook --ask-become-pass install_apache.yml`. After executing this command, you may notice that it did not become successful in the Centos VM. You can see that the Centos VM has `failed=1`. Only the two remote servers have been changed. The reason is that Centos VM does not support "apt" as the package manager. The default package manager for Centos is "yum."

```
PLAY RECAP *****
*****
centos_vm      : ok=0    changed=0    unreachable
=0    failed=1    skipped=0    rescued=0    ignored=0
server1       : ok=4    changed=1    unreachable
=0    failed=0    skipped=0    rescued=0    ignored=0
server2       : ok=4    changed=1    unreachable
=0    failed=0    skipped=0    rescued=0    ignored=0
```

3. Edit the `install_apache.yml` file and insert the lines shown below.

Make sure to save the file and exit.

Run `ansible-playbook --ask-become-pass install_apache.yml` and describe the

result.

If you have a mix of Debian and Ubuntu servers, you can change the configuration of your playbook like this.

- name: update repository index

apt:

update_cache: yes

when: ansible_distribution in ["Debian", "Ubuntu"]

Note: This will work also if you try. Notice the changes are highlighted.

```

jeraldinterno@localhost:~
GNU nano 7.2      install_apache.yml
--
- name: Install Apache and PHP
  hosts: all
  become: yes

  tasks:
    - name: update repository index
      apt:
        update_cache: yes
        when: ansible_distribution in ["Debian", "Ubuntu"]

    - name: install apache2 package
      apt:
        name: apache2
        state: present
        when: ansible_distribution in ["Debian", "Ubuntu"]

    - name: add PHP support for apache
      apt:
        name: libapache2-mod-php
        state: present
        when: ansible_distribution in ["Debian", "Ubuntu"]
  
```

```

jerald@workstation:~/ansible-lab$ ansible-playbook -i inventory.yaml --ask-become-pass install_apache.yml
BECOME password:

PLAY [Install Apache and PHP] *****

TASK [Gathering Facts] *****
ok: [server1]
ok: [server2]
ok: [centos_vm]

TASK [update repository index] *****
skipping: [centos_vm]
changed: [server2]
changed: [server1]

TASK [install apache2 package] *****
skipping: [centos_vm]
ok: [server1]
ok: [server2]

TASK [add PHP support for apache] *****
skipping: [centos_vm]
ok: [server1]
ok: [server2]

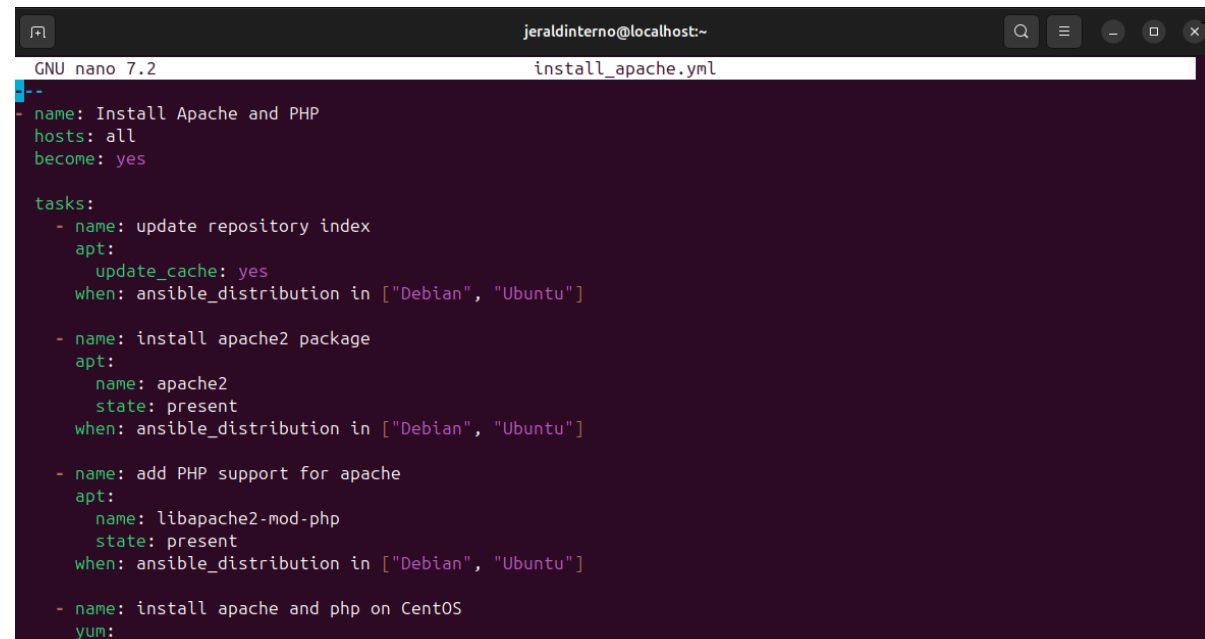
PLAY RECAP *****
centos_vm      : ok=1    changed=0    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
server1        : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server2        : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

```

4. Edit the `install_apache.yml` file and insert the lines shown below.

Make sure to save and exit.

Run `ansible-playbook --ask-become-pass install_apache.yml` and describe the Result.



```

jeraldinterno@localhost:~
GNU nano 7.2 install_apache.yml
--
- name: Install Apache and PHP
  hosts: all
  become: yes

  tasks:
    - name: update repository index
      apt:
        update_cache: yes
        when: ansible_distribution in ["Debian", "Ubuntu"]

    - name: install apache2 package
      apt:
        name: apache2
        state: present
        when: ansible_distribution in ["Debian", "Ubuntu"]

    - name: add PHP support for apache
      apt:
        name: libapache2-mod-php
        state: present
        when: ansible_distribution in ["Debian", "Ubuntu"]

    - name: install apache and php on CentOS
      yum:

```

```
name:
  - httpd
  - php
state: present
when: ansible_distribution == "CentOS"
```

```

jeraldinterno@localhost:~
TASK [Gathering Facts] *****
ok: [server1]
ok: [server2]
ok: [centos_vm]

TASK [update repository index] *****
skipping: [centos_vm]
changed: [server1]
changed: [server2]

TASK [install apache2 package] *****
skipping: [centos_vm]
ok: [server1]
ok: [server2]

TASK [add PHP support for apache] *****
skipping: [centos_vm]
ok: [server2]
ok: [server1]

TASK [install apache and php on CentOS] *****
skipping: [server1]
skipping: [server2]
changed: [centos_vm]

PLAY RECAP *****
centos_vm      : ok=2    changed=1    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
server1       : ok=4    changed=1    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
server2       : ok=4    changed=1    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
  
```

- Ansible dynamically evaluated host facts and executed apt for Debian/Ubuntu hosts while running yum on centos_vm. All managed nodes finished successfully with failed=0.

5. To verify the installations, go to CentOS VM and type its IP address on the browser. Was it successful? The answer is no. It's because the httpd service or the Apache HTTP server in the CentOS is not yet active. Thus, you need to activate it first.

```

[jeraldinterno@localhost ~]$ systemctl status httpd
○ httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: d
   Drop-In: /usr/lib/systemd/system/httpd.service.d
           └─php-fpm.conf
   Active: inactive (dead)
   Docs: man:httpd.service(8)

lines 1-6/6 (END)
  
```

5.1 To activate, go to the CentOS VM terminal and enter the following:

systemctl status httpd

The result of this command tells you that the service is inactive.

5.2 Issue the following command to start the service:

sudo systemctl start httpd

(When prompted, enter the sudo password)

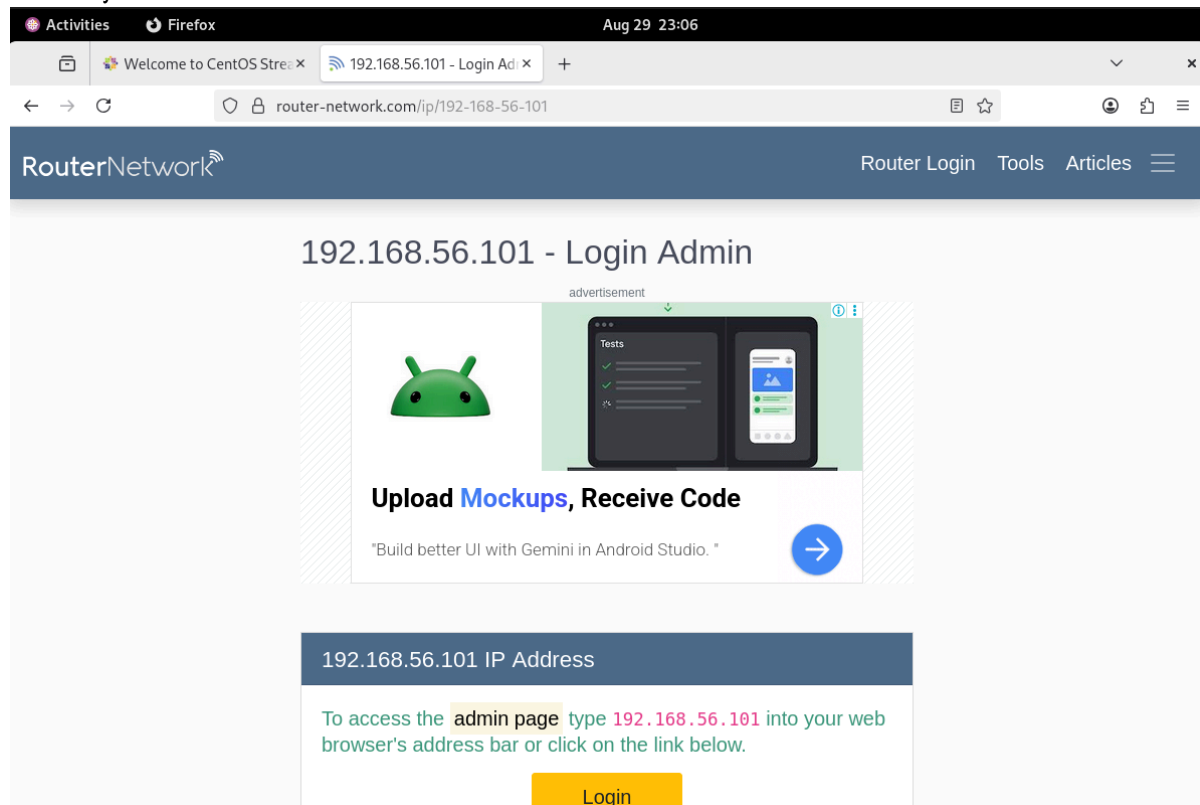
sudo firewall-cmd --add-port=80/tcp

(The result should be a success)

```
[jeraldinterno@localhost ~]$ sudo systemctl start httpd
[sudo] password for jeraldinterno:
[jeraldinterno@localhost ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[jeraldinterno@localhost ~]$ sudo firewall-cmd --add-port=80/tcp --permanent
success
[jeraldinterno@localhost ~]$ sudo firewall-cmd --reload
success
```

5.3 To verify the service is already running, go to CentOS VM and type its IP address on the browser. Was it successful? (Screenshot the browser)

- yes



Task 2: Refactoring playbook

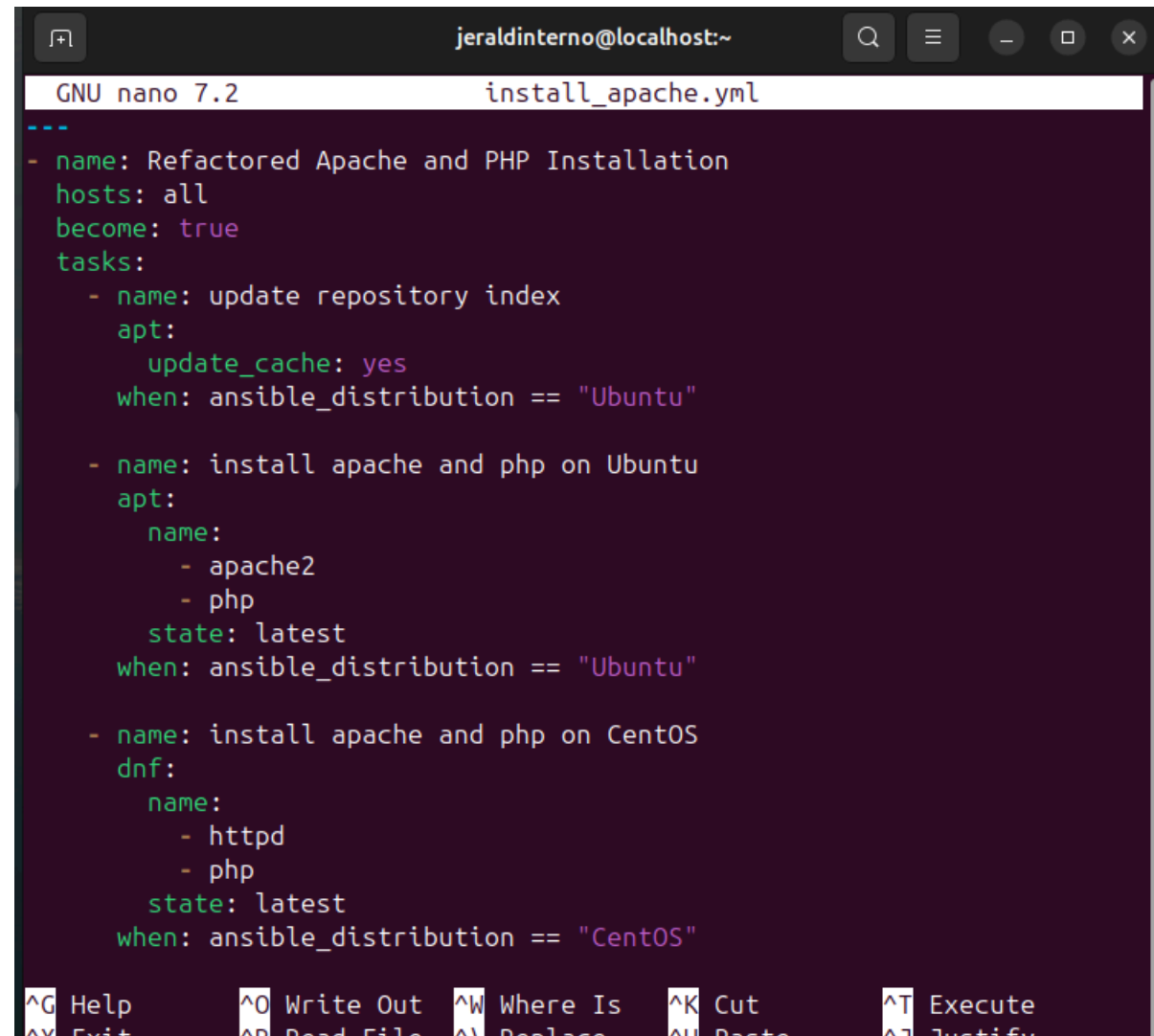
This time, we want to make sure that our playbook is efficient and that the codes are easier to read. This will also makes run ansible more quickly if it has to execute fewer tasks to do the same thing.

1. Edit the playbook install_apache.yml. Currently, we have three tasks targeting our Ubuntu machines and 3 tasks targeting our CentOS machine. Right now, we try to consolidate some tasks that are typically the same. For example, we can

consolidate two plays that install packages. We can do that by creating a list of installation packages as shown below:

Make sure to save the file and exit.

```
install_apache.yml: command not found
jerald@workstation:~/ansible-lab$ nano install_apache.yml
```



```
GNU nano 7.2 install_apache.yml
---
- name: Refactored Apache and PHP Installation
  hosts: all
  become: true
  tasks:
    - name: update repository index
      apt:
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: install apache and php on Ubuntu
      apt:
        name:
          - apache2
          - php
        state: latest
      when: ansible_distribution == "Ubuntu"

    - name: install apache and php on CentOS
      dnf:
        name:
          - httpd
          - php
        state: latest
      when: ansible_distribution == "CentOS"

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Run ansible-playbook --ask-become-pass install_apache.yml and describe the Result.

```
jerald@workstation:~/ansible-lab$ ansible-playbook -i inventory.yaml -
--ask-become-pass install_apache.yml
BECOME password:
```

```

TASK [update repository index] *****
*****
skipping: [centos_vm]
changed: [server1]
changed: [server2]

TASK [install apache and php on Ubuntu] *****
*****
skipping: [centos_vm]
changed: [server2]
changed: [server1]

TASK [install apache and php on CentOS] *****
Settings **
skipping: [server1]
skipping: [server2]
ok: [centos_vm]

PLAY RECAP *****
*****
centos_vm      : ok=2    changed=0    unreachable=0    fai
led=0    skipped=2    rescued=0    ignored=0
server1       : ok=3    changed=2    unreachable=0    fai
led=0    skipped=1    rescued=0    ignored=0
server2       : ok=3    changed=2    unreachable=0    fai
led=0    skipped=1    rescued=0    ignored=0

```

- The playbook runs successfully with failed=0.

2. Edit the playbook install_apache.yml again. In task 2.1, we consolidated the plays into one play. This time we can actually consolidated everything in just 2 plays. This can be done by removing the update repository play and putting the command update_cache: yes below the command state: latest. See below for reference:

Make sure to save the file and exit.

Run `ansible-playbook --ask-become-pass install_apache.yml` and describe the Result.

```

gerald@workstation:~/ansible-lab$ ansible-playbook -i inventory.yaml --ask-become-pass install_apache.yml
BECOME password:

PLAY [Install Apache and PHP] *****

TASK [Gathering Facts] *****
ok: [server1]
ok: [server2]
ok: [centos_vm]

TASK [install apache and php on Ubuntu] *****
skipping: [centos_vm]
ok: [server1]
ok: [server2]

TASK [install apache and php on CentOS] *****
skipping: [server1]
skipping: [server2]
ok: [centos_vm]

PLAY RECAP *****
centos_vm      : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
server1       : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
server2       : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0

```

```
GNU nano 7.2                                install_apache.yml
--
- name: Install Apache and PHP
  hosts: all
  become: true
  tasks:
    - name: install apache and php on Ubuntu
      apt:
        name:
          - apache2
          - php
        state: latest
        update_cache: yes
      when: ansible_distribution == "Ubuntu"

    - name: install apache and php on CentOS
      dnf:
        name:
          - httpd
          - php
        state: latest
      when: ansible_distribution == "CentOS"
```

- The playbook executes in 2 conditional tasks under a single play. Ansible checks OS facts and successfully installs Apache and PHP on Ubuntu (server1, server2) and CentOS (centos_vm), skipping inapplicable tasks per host with failed=0.

3. Finally, we can consolidate these 2 plays in just 1 play. This can be done by declaring variables that will represent the packages that we want to install. Basically, the `apache_package` and `php_package` are variables. The names are arbitrary, which means we can choose different names. We also take out the line `when: ansible_distribution`. Edit the playbook `install_apache.yml` again and make sure to follow the below image. Make sure to save the file and exit. Run `ansible-playbook --ask-become-pass install_apache.yml` and describe the Result.

```

jeraldinterno@localhost:~
GNU nano 7.2                                install_apache.yml
---
- name: Install Apache and PHP
  hosts: all
  become: true
  tasks:
    - name: install web server packages
      apt:
        name:
          - "{{ apache_package }}"
          - "{{ php_package }}"
        state: latest
        update_cache: yes

jerald@workstation:~/ansible-lab$ ansible-playbook -i inventory.yaml --ask-become-pass install_apache.yml
BECOME password:

PLAY [Install Apache and PHP] *****

TASK [Gathering Facts] *****
ok: [server2]
ok: [server1]
ok: [centos_vm]

TASK [install web server packages] *****
[WARNING]: Updating cache and auto-installing missing dependency: python3-apt
fatal: [centos_vm]: FAILED! => {"changed": false, "cmd": "apt-get update", "msg": "[Errno 2] No such file or di
b'apt-get'", "rc": 2, "stderr": "", "stderr_lines": [], "stdout": "", "stdout_lines": []}
ok: [server1]
ok: [server2]

PLAY RECAP *****
centos_vm      : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignore=
server1       : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignore=
server2       : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignore=

```

- Ansible throws an undefined variable error because `apache_package` and `php_package` are not yet defined in the inventory. Also, the running the `apt` module on `centos_vm` throws a module failure error because CentOS does not support `apt`

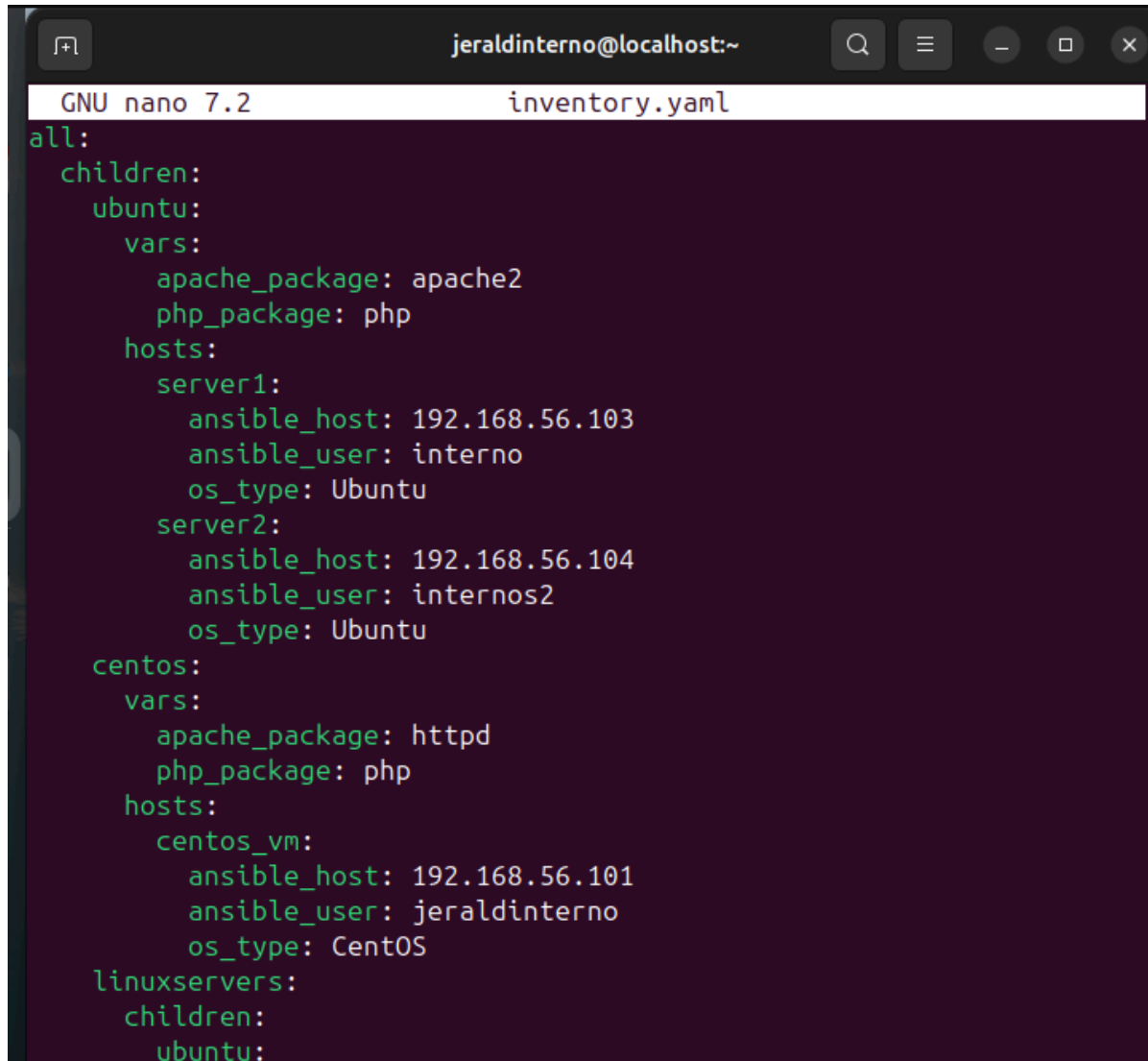
4. Unfortunately, task 2.3 was not successful. It's because we need to change something in the inventory file so that the variables we declared will be in place. Edit the inventory file and follow the below configuration:

Make sure to save the inventory file and exit.

Finally, we still have one more thing to change in our `install_apache.yml` file. In task 2.3, you may notice that the package is assign as `apt`, which will not run in CentOS. Replace the `apt` with `package`. `Package` is a module in ansible that is generic, which is going to use whatever package manager the underlying host or the target server uses. For Ubuntu it will automatically use `apt`, and for CentOS it will automatically use `dnf`. Make sure to save the file and exit. For more details about the ansible package, you may refer to this documentation: [ansible.builtin.package – Generic OS package manager — Ansible](#)

Documentation

Run `ansible-playbook --ask-become-pass install_apache.yml` and describe the Result.

A screenshot of a terminal window titled 'jeraldinterno@localhost:~'. The window shows the 'inventory.yml' file being edited with 'GNU nano 7.2'. The file content is as follows:

```
all:
  children:
    ubuntu:
      vars:
        apache_package: apache2
        php_package: php
      hosts:
        server1:
          ansible_host: 192.168.56.103
          ansible_user: interno
          os_type: Ubuntu
        server2:
          ansible_host: 192.168.56.104
          ansible_user: internos2
          os_type: Ubuntu
    centos:
      vars:
        apache_package: httpd
        php_package: php
      hosts:
        centos_vm:
          ansible_host: 192.168.56.101
          ansible_user: jeraldinterno
          os_type: CentOS
  linuxservers:
    children:
      ubuntu:
```

```

jeraldinterno@localhost:~
GNU nano 7.2 install_apache.yml
--
- name: Install Apache and PHP
  hosts: all
  become: true
  tasks:
    - name: install web server packages
      package:
        name:
          - "{{ apache_package }}"
          - "{{ php_package }}"
        state: latest

jerald@workstation:~/ansible-lab$ ansible-playbook -i inventory.yaml --ask-become-pass install_apache.yml
BECOME password:

PLAY [Install Apache and PHP] *****

TASK [Gathering Facts] *****
ok: [server1]
ok: [server2]
ok: [centos_vm]

TASK [install web server packages] *****
ok: [server1]
ok: [server2]
ok: [centos_vm]

PLAY RECAP *****
centos_vm      : ok=2    changed=0    unreachable=0    failed=0    skipped=0
rescued=0     ignored=0
server1       : ok=2    changed=0    unreachable=0    failed=0    skipped=0
rescued=0     ignored=0
server2       : ok=2    changed=0    unreachable=0    failed=0    skipped=0
rescued=0     ignored=0
  
```

- Execution completes successfully with failed=0 on all target hosts. The generic package module automatically routes package management through apt for Ubuntu hosts (installing apache2 and php) and dnf for CentOS hosts (installing httpd and php) within a single, unified Ansible task.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Reflections:

Answer the following:

1. Why do you think refactoring of playbook codes is important?
 - Refactoring reduces code duplication, improves readability, and makes playbooks easier to maintain. By using variables and generic modules (like `package`), you can execute tasks across different operating systems with cleaner, modular, and scalable code without changing the playbook's actual functionality.
2. When do we use the "when" command in playbook?
 - Use the when clause in Ansible when you need to execute or skip a task based on specific conditions—such as target host facts (`ansible_distribution == "Ubuntu"`), environment variables, command outputs, or custom boolean conditions.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.